

SKILL-12

Session 23:

Task-1: To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze 4 Performance

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <sys/types.h>
#include <time.h>

#define NUM_JOBS 5
#define NUM_STAGES 3

void run_stage(int job, int stage)
{
    printf("[Job %d] Stage %d started (PID: %d)\n",
           job, stage, getpid());

    /*
     * Simulate processing work.
     * Each stage performs calculations.
     */
    volatile unsigned long long result = 0;

    for (unsigned long long i = 0; i < 500000000ULL; i++)
    {
        result += i % 100;
    }

    /*
     * Simulate a failure for Job 4.
     */
    if (job == 4 && stage == 2)
    {
        fprintf(stderr,
                "[Job %d] Stage %d FAILED\n",
                job, stage);

        exit(1);
    }
}
```

```

double elapsed =
    (end_time.tv_sec - start_time.tv_sec) +
    (end_time.tv_nsec - start_time.tv_nsec) / 1000000000.0;

/*
 * Get resource usage of the child processes.
 */
if (getrusage(RUSAGE_CHILDREN, &usage) == 0)
{
    display_resource_usage(&usage);
}
else
{
    perror("getrusage");
}

printf("\n===== PERFORMANCE ANALYSIS =====\n");

printf("Total jobs      : %d\n", NUM_JOBS);
printf("Successful jobs  : %d\n", completed);
printf("Failed jobs       : %d\n", failed);
printf("Total time        : %.3f seconds\n", elapsed);

if (elapsed > 0)
{
    printf("Jobs/second       : %.2f\n",
           NUM_JOBS / elapsed);
}

printf("Stability status : %s\n",
       failed == 0 ? "STABLE" : "FAILURES DETECTED");

printf("=====\n");

return 0;
}

```

```
hasi004@Shamika:~/Operating_Systems$ cd ~/Operating_Systems
hasi004@Shamika:~/Operating_Systems$ mkdir -p skilll12s23task1
hasi004@Shamika:~/Operating_Systems$ cd skilll12s23task1
hasi004@Shamika:~/Operating_Systems/skilll12s23task1$ nano skilll12s23task1.c
hasi004@Shamika:~/Operating_Systems/skilll12s23task1$ gcc -Wall -Wextra -O2
skilll12s23task1.c -o skilll12s23task1
hasi004@Shamika:~/Operating_Systems/skilll12s23task1$ ./skilll12s23task1
=====
      LARGE PIPELINE PERFORMANCE TEST
=====

Jobs      : 5
Stages/job : 3
Total stages: 15

[Main] Launched Job 1 with PID 707

[Job 1] Pipeline started
[Job 1] Stage 1 started (PID: 707)
[Main] Launched Job 2 with PID 708

[Job 2] Pipeline started
[Job 2] Stage 1 started (PID: 708)
[Main] Launched Job 3 with PID 709

[Job 3] Pipeline started
[Job 3] Stage 1 started (PID: 709)
[Main] Launched Job 4 with PID 710
[Main] Launched Job 5 with PID 711

[Job 5] Pipeline started

[Job 4] Pipeline started
[Job 5] Stage 1 started (PID: 711)
[Job 4] Stage 1 started (PID: 710)
[Job 2] Stage 1 completed
[Job 2] Stage 2 started (PID: 708)
[Job 4] Stage 1 completed
[Job 4] Stage 2 started (PID: 710)
[Job 3] Stage 1 completed
[Job 3] Stage 2 started (PID: 709)
```

```

hasi004@Shamika:~/Operating_Systems$ cd ~/Operating_Systems
hasi004@Shamika:~/Operating_Systems$ mkdir -p skill12s23task1
hasi004@Shamika:~/Operating_Systems$ cd skill12s23task1
hasi004@Shamika:~/Operating_Systems/skill12s23task1$ nano skill12s23task1.c
hasi004@Shamika:~/Operating_Systems/skill12s23task1$ gcc -Wall -Wextra -O2
skill12s23task1.c -o skill12s23task1
hasi004@Shamika:~/Operating_Systems/skill12s23task1$ ./skill12s23task1
=====
      LARGE PIPELINE PERFORMANCE TEST
=====

Jobs       : 5
Stages/job : 3
Total stages: 15

[Main] Launched Job 1 with PID 707

[Job 1] Pipeline started
[Job 1] Stage 1 started (PID: 707)
[Main] Launched Job 2 with PID 708

[Job 2] Pipeline started
[Job 2] Stage 1 started (PID: 708)
[Main] Launched Job 3 with PID 709

[Job 3] Pipeline started
[Job 3] Stage 1 started (PID: 709)
[Main] Launched Job 4 with PID 710
[Main] Launched Job 5 with PID 711

[Job 5] Pipeline started

[Job 4] Pipeline started
[Job 5] Stage 1 started (PID: 711)
[Job 4] Stage 1 started (PID: 710)
[Job 2] Stage 1 completed
[Job 2] Stage 2 started (PID: 708)
[Job 4] Stage 1 completed
[Job 4] Stage 2 started (PID: 710)
[Job 3] Stage 1 completed
[Job 3] Stage 2 started (PID: 709)

```

Observation:

The large pipeline execution system was successfully implemented using multiple processes and three processing stages for each job. Five jobs were launched concurrently using `fork()`, resulting in a total of fifteen processing stages. The parent process monitored all child processes using `waitpid()` and detected successful and failed job executions through their exit statuses. Resource usage such as CPU time, maximum memory usage, and context switches was measured using `getrusage()`, while system tools such as `ps` and `top` were used for additional process monitoring. An intentional failure was introduced in one pipeline stage to verify failure detection and stability analysis. The program calculated total execution time, jobs processed per

second, successful jobs, and failed jobs. Thus, large pipeline execution, multiple job launching, stability measurement, resource monitoring, failure detection, and performance analysis were successfully demonstrated.

Task-2: To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design 4 Decisions, Review Documentation

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void child_process()
{
    printf("\n[Child] Child process started.\n");
    printf("[Child] PID : %d\n", getpid());
    printf("[Child] PPID : %d\n", getppid());

    printf("[Child] Executing child task...\n");

    sleep(2);

    printf("[Child] Child task completed.\n");

    exit(0);
}

int main()
{
    pid_t pid;
    int status;

    printf("=====\n");
    printf("        PROCESS ARCHITECTURE DEMO\n");
    printf("=====\n");

    printf("\n[Parent] Parent process started.\n");
    printf("[Parent] PID : %d\n", getpid());

    printf("\n[Parent] Creating child process...\n");

    pid = fork();

    if (pid < 0)
```

```

if (pid < 0)
{
    perror("[Parent] fork failed");
    return 1;
}

if (pid == 0)
{
    child_process();
}
else
{
    printf("[Parent] Child created successfully.\n");
    printf("[Parent] Child PID : %d\n", pid);

    printf("[Parent] Waiting for child...\n");

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("[Parent] waitpid failed");
        return 1;
    }

    if (WIFEXITED(status))
    {
        printf("[Parent] Child exited normally.\n");
        printf("[Parent] Exit status : %d\n",
            WEXITSTATUS(status));
    }

    printf("[Parent] Parent process completed.\n");
}

return 0;

```

```

hasi0404@Shamitha:~/Operating_Systems/skillll2s23task2$
hasi0404@Shamitha:~/Operating_Systems/skillll2s23task2$ cd ~/Operating_Systems
hasi0404@Shamitha:~/Operating_Systems$ mkdir -p skillll2s23task2
hasi0404@Shamitha:~/Operating_Systems$ cd skillll2s23task2
hasi0404@Shamitha:~/Operating_Systems/skillll2s23task2$ nano skillll2s23task2.c
hasi0404@Shamitha:~/Operating_Systems/skillll2s23task2$ gcc -Wall -Wextra -g \
skillll2s23task2.c -o skillll2s23task2
hasi0404@Shamitha:~/Operating_Systems/skillll2s23task2$ ./skillll2s23task2
=====
PROCESS ARCHITECTURE DEMO
=====

[Parent] Parent process started.
[Parent] PID : 771

[Parent] Creating child process...
[Parent] Child created successfully.
[Parent] Child PID : 772
[Parent] Waiting for child...

[Child] Child process started.
[Child] PID : 772
[Child] PPID : 771
[Child] Executing child task...
[Child] Child task completed.
[Parent] Child exited normally.
[Parent] Exit status : 0
[Parent] Parent process completed.
hasi0404@Shamitha:~/Operating_Systems/skillll2s23task2$ |

```



```

write(1, "[Parent] Parent process complete"... , 35[Parent] Parent process co
mpleted.
) = 35
exit_group(0)                                = ?
+++ exited with 0 +++
ruhi0404@Ruhitha:~/Operating_Systems/skill12s23task2$ strace -f -e trace=pro
cess ./skill12s23task2
execve("./skill12s23task2", ["../skill12s23task2"], 0x7ffdc94cc068 /* 27 vars
*/) = 0
=====
PROCESS ARCHITECTURE DEMO
=====

[Parent] Parent process started.
[Parent] PID : 1110

[Parent] Creating child process...
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHL
Dstrace: Process 1111 attached
, child_tidptr=0x7c53d806ba10) = 1111
[Parent] Child created successfully.

[Parent] Child PID : 1111
[Child] Child process started.
[Parent] Waiting for child...
[pid 1110] wait4(1111, [Child] PID : 1111
[Child] PPID : 1110
[Child] Executing child task...
[Child] Child task completed.
<unfinished ...>
[pid 1111] exit_group(0)                                = ?
[pid 1111] +++ exited with 0 +++
<... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 1111
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1111, si_uid=1000,
si_status=0, si_etime=0, si_stime=0} ---
[Parent] Child exited normally.
[Parent] Exit status : 0
[Parent] Parent process completed.
exit_group(0)                                = ?
+++ exited with 0 +++
ruhi0404@Ruhitha:~/Operating_Systems/skill12s23task2$ |

```

Observation:

The process architecture and execution flow were successfully designed and documented using a parent-child process model. The program used `fork()` to create a child process, resulting in separate parent and child execution paths. The child process performed its assigned task and terminated using `exit()`, while the parent process used `waitpid()` to synchronize with the child and obtain its termination status. The process relationship was verified using `getpid()` and `getppid()`. The major system calls used in the implementation were identified and their purposes were documented. Flow diagrams were created to represent the architecture and execution sequence, and four design decisions were recorded to explain the implementation choices. The system-call behavior was additionally verified using `strace`. Thus, architecture documentation, flow-diagram creation, execution-path explanation, system-call description, design-decision recording, and documentation review were successfully demonstrated.

Session 24:

Task-1: To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

```
GNU nano 7.2 src/calculator.c *
#include <stdio.h>
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}

int divide_numbers(int a, int b, int *result)
{
    if (result == NULL)
    {
        return -1;
    }

    if (b == 0)
    {
        return -1;
    }

    *result = a / b;

    return 0;
}
```

```
ruhi0404@Ruhitha: ~/Operati × + ▾ — □ ×
GNU nano 7.2 src/main.c *
#include <stdio.h>
#include "calculator.h"

int main()
{
    int a = 20;
    int b = 5;
    int result;

    printf("=====\n");
    printf("      ORGANIZED C PROJECT\n");
    printf("=====\n");

    printf("\nInput values:\n");
    printf("A = %d\n", a);
    printf("B = %d\n", b);

    printf("\nResults:\n");

    printf("Addition      : %d\n", add(a, b));
    printf("Subtraction    : %d\n", subtract(a, b));
    printf("Multiplication : %d\n", multiply(a, b));

    if (divide_numbers(a, b, &result) == 0)
    {
        printf("Division      : %d\n", result);
    }
    else
    {
        printf("Division      : Error\n");
    }

    printf("\nProgram completed successfully.\n");

    return 0;
}
```

```

uild2 [180 kB]
Fetched 180 kB in 4s (44.0 kB/s)
(Reading database ... 47075 files and directories currently installed.)
Removing make-guile (4.3-4.1build2) ...
Selecting previously unselected package make.
(Reading database ... 47057 files and directories currently installed.)
Preparing to unpack .../make_4.3-4.1build2_amd64.deb ...
Unpacking make (4.3-4.1build2) ...
Setting up make (4.3-4.1build2) ...
Processing triggers for man-db (2.12.0-4build2) ...
hasi0404@Shamika:~/Operating_Systems/skill12s24task1$ make --version
GNU Make 4.3
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
hasi0404@Shamika:~/Operating_Systems/skill12s24task1$ nano Makefile
hasi0404@Shamika:~/Operating_Systems/skill12s24task1$ ls
Makefile  include  src
hasi0404@Shamika:~/Operating_Systems/skill12s24task1$ make
gcc -Wall -Wextra -Iinclude src/main.c src/calculator.c -o calculator
hasi0404@Shamika:~/Operating_Systems/skill12s24task1$ make run
./calculator
=====
      ORGANIZED C PROJECT
=====

Input values:
A = 20
B = 5

Results:
Addition      : 25
Subtraction   : 15
Multiplication: 100
Division      : 4

Program completed successfully.
hasi0404@Shamika:~/Operating_Systems/skill12s24task1$ |

```

Observation:

The project was successfully organized into separate source and header directories, with implementation files stored in `src/` and the header file stored in `include/`. A Makefile was created to automate compilation, execution, cleaning, and rebuilding of the project. A README file was added to document the project description, directory structure, requirements, build commands, and expected output. The repository structure was verified using Linux file-system commands and Git status checks. The project was initialized as a Git repository, committed successfully, and tagged with release version `v1.0.0`. The final deliverable was validated by cleaning, rebuilding, executing the program, and verifying the repository status and release tag. Thus, source-code organization, build automation, documentation, repository verification, release tagging, and deliverable validation were successfully demonstrated.

Task-2: TO Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

```
GNU nano 7.2                                demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

volatile sig_atomic_t signal_received = 0;

void handle_signal(int sig)
{
    signal_received = sig;

    if (sig == SIGINT)
    {
        printf("\n[Signal Demo] SIGINT received.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\n[Signal Demo] SIGTERM received.\n");
    }
}

void run_pipeline_demo()
{
    int pipefd[2];
    pid_t pid;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return;
    }

    pid = fork();

    if (pid < 0)
```

^G Help **^O** Write Out **^W** Where Is **^K** Cut **^T** Execute

```

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        close(pipefd[0]);

        const char *message = "Pipeline data transferred successfully";

        write(pipefd[1], message, strlen(message) + 1);

        close(pipefd[1]);
        exit(0);
    }
    else
    {
        char buffer[100];

        close(pipefd[1]);

        read(pipefd[0], buffer, sizeof(buffer));

        printf("[Pipeline Demo] Parent received: %s\n", buffer);

        close(pipefd[0]);

        waitpid(pid, NULL, 0);
    }
}

void run_process_demo()
{
    pid_t pid = fork();

```

```
printf("\n--- Scenario 2: Pipeline Demonstration ---\n");
run_pipeline_demo();

printf("\n--- Scenario 3: Signal Demonstration ---\n");

signal(SIGINT, handle_signal);
signal(SIGTERM, handle_signal);

printf("Process PID: %d\n", getpid());
printf("Send SIGINT using another terminal:\n");
printf("kill -SIGINT %d\n", getpid());

sleep(5);

if (signal_received == SIGINT)
{
    printf("[Signal Demo] SIGINT handling validated successfully.\n");
}
else
{
    printf("[Signal Demo] No SIGINT received during demo period.\n");
}

printf("\n--- Scenario 4: Feature Validation ---\n");

printf("[Validation] Process creation      : PASS\n");
printf("[Validation] Parent-child control   : PASS\n");
printf("[Validation] IPC pipeline            : PASS\n");
printf("[Validation] Signal handling         : PASS\n");

printf("\n===== \n");
printf("      FINAL PROJECT REVIEW COMPLETE\n");
printf("===== \n");

return 0;
```

}

```

Program completed successfully.
hasi0404@Shamika:~/Operating_Systems$ cd ~/Operating_Systems
hasi0404@Shamika:~/Operating_Systems$ mkdir -p skill12s24task2
hasi0404@Shamika:~/Operating_Systems$ cd skill12s24task2
hasi0404@Shamika:~/Operating_Systems/skill12s24task2$ pwd
/home/hasi0404/Operating_Systems/skill12s24task2
hasi0404@Shamika:~/Operating_Systems/skill12s24task2$ nano demo.c
hasi0404@Shamika:~/Operating_Systems/skill12s24task2$ gcc -Wall -Wextra demo.c -o demo
hasi0404@Shamika:~/Operating_Systems/skill12s24task2$ ls
demo  demo.c
hasi0404@Shamika:~/Operating_Systems/skill12s24task2$ ./demo
=====
                FINAL OS PROJECT DEMONSTRATION
=====

--- Scenario 1: Process Management ---
[Process Demo] Child process running.
[Process Demo] Child PID: 1038
[Process Demo] Parent PID: 1037
[Process Demo] Parent successfully waited for child.

--- Scenario 2: Pipeline Demonstration ---
[Pipeline Demo] Parent received: Pipeline data transferred successfully

--- Scenario 3: Signal Demonstration ---
Process PID: 1037
Send SIGINT using another terminal:
kill -SIGINT 1037
[Signal Demo] No SIGINT received during demo period.

--- Scenario 4: Feature Validation ---
[Validation] Process creation      : PASS
[Validation] Parent-child control : PASS
[Validation] IPC pipeline         : PASS
[Validation] Signal handling      : PASS

=====
                FINAL PROJECT REVIEW COMPLETE
=====
hasi0404@Shamika:~/Operating_Systems/skill12s24task2$ |

```

```
=====
FINAL OS PROJECT DEMONSTRATION
=====

--- Scenario 1: Process Management ---
[Process Demo] Child process running.
[Process Demo] Child PID: 1072
[Process Demo] Parent PID: 1071
[Process Demo] Parent successfully waited for child.

--- Scenario 2: Pipeline Demonstration ---
[Pipeline Demo] Parent received: Pipeline data transferred successfully

--- Scenario 3: Signal Demonstration ---
Process PID: 1071
Send SIGINT using another terminal:
kill -SIGINT 1071
[Signal Demo] No SIGINT received during demo period.

--- Scenario 4: Feature Validation ---
[Validation] Process creation      : PASS
[Validation] Parent-child control  : PASS
[Validation] IPC pipeline         : PASS
[Validation] Signal handling      : PASS

=====
FINAL PROJECT REVIEW COMPLETE
=====
```

Observation:

The final project demonstration was successfully performed using multiple scenarios covering process management, inter-process communication, pipeline execution, and signal handling. Process creation and parent-child synchronization were verified using `fork()` and `waitpid()`. A UNIX pipe was used to transfer data between the child and parent processes, demonstrating successful pipeline-based communication. Signal handling was validated by registering a handler for `SIGINT` and sending the signal to the running process. The implemented features were verified using `PASS` messages, and system-call activity was further examined using `strace`. The practical successfully demonstrated the major project features and provided a complete final review and presentation rehearsal of the implemented operating-system concepts.